

从任务记录工具，到个人工作操作系统

核心不是“管理更多事项”，而是把今天该干什么、现在该干什么、做完留下什么知识串成闭环。



三层产品结构：执行、知识、智能

近期不要把 Chronicle 做成重型项目管理软件。先围绕你的真实工作习惯，建立三个能每天复用的支柱。

每天盯着用

Today Focus

- 像记事本一样编辑
- 10-20 分钟工作块
- 当前项高亮和倒计时
- 顺延、跳过、完成、重排
- 降低启动阻力

长期留下来

Notes

- 从 task log 提炼知识
- 项目、决策、手册、复盘
- 任务和日志反向链接
- 统一搜索和回顾
- 减少重复理解成本

帮你做转换

Local LLM

- 生成今日计划
- 重排剩余时间
- 任务拆解成小块
- 总结任务成为 note
- 日报、周报、回顾

近期最应该先做：今日工作纸条

它不是 plan board 的增强版，而是一个能对抗拖延和注意力漂移的外置执行脚本。

现在正在做

10:15-10:35

写 data integrity test

完成当前

顺延 10m

我卡住了

记录到日志

09:30-09:45 整理昨天遗留事项

09:45-10:05 看 searchService diff

10:05-10:15 休息

10:15-10:35 写 data integrity test

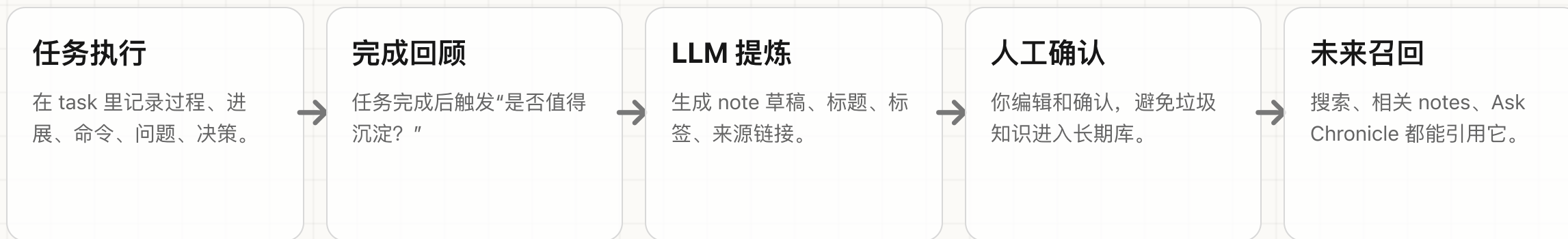
10:35-10:50 跑测试并记录结果

10:50-11:10 总结成 task log

设计原则：UI 像文本，数据可结构化；用户只需要直接改字，不需要进入“管理系统模式”。

Notes 不是替代日志，而是把日志沉淀成长期知识

task/log 保留原始工作痕迹；note 承载未来还会复用的信息、判断和流程。



关键闭环：工作记录 → 知识提炼 → 来源可追溯 → 后续任务可复用。

LLM 策略：API-first，加内置高频 workflow

Chronicle 不应该只是接一个聊天框。它应该先稳定数据和 API，再把 LLM 嵌入具体动作。

内置 LLM workflow

Plan Today / Replan / Summarize / Extract Note

Chronicle UI

Board / Today / Notes / Reports / Planner

Chronicle API Core

任务、日志、计划、Notes、搜索、报表都通过同一套本地 API 暴露。

本地 LLM 服务

OpenAI-compatible / Ollama / Custom endpoint

外部 Agent / MCP

高级 prompt、自动化、跨工具 workflow

结论：常用动作做进 Chronicle；探索性自动化交给外部 agent。两者共享同一套 API。

中期 Planner：先做可执行周计划，再谈 Gantt

ETA、deadline、planned date 是基础。Gantt 是后续视图，不是第一步。

任务	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Risk
Search index	2h implementation							正常
Notes MVP		6h CRUD + links						正常
Day Script	4h raw text + highlight							优先
Release work				deadline Friday			紧张	

Planner 的核心问题：今天和本周是否排得下？什么会延期？现在应该先做哪一个？

路线图：先恢复日常使用，再扩展成工作系统

眼下立刻落地

近期

- 1 Day Script raw text, Today 页面直接编辑
- 2 解析时间段, 高亮当前 block 和倒计时
- 3 完成、跳过、顺延、记录到日志
- 4 LLM 生成今日工作纸条草稿

1-3 个月

中期

- 1 Notes CRUD、标签、任务和日志链接
- 2 Task/log → Note 的 LLM 草稿 workflow
- 3 ETA、deadline、planned date
- 4 周计划视图和重排剩余时间

Think Big

远期

- 1 个人工作操作系统：执行、知识、复盘闭环
- 2 本地 LLM 助理和外部 Agent/MCP
- 3 Timeline/Gantt、依赖、过载风险提示
- 4 个人工作模式分析和长期召回